

COVERAGE_IMPROVEMENT_PLAN

Test Coverage Improvement Plan

Current Coverage: 77.0% **Target Coverage:** 80%+ **Date Created:** 2025-11-07

Executive Summary

This document outlines the plan to improve test coverage from the current 77% to the target of 80%+ for the HelixCode project. The plan focuses on adding tests for low-coverage areas that don't require external dependencies.

Low-Coverage Areas Identified

Priority 1: Config Package (75.7% coverage)

File: internal/config/config.go

Low-Coverage Functions: 1. Load - 65.6% coverage 2. findConfigFile - 33.3% coverage 3. validateConfig - 66.7% coverage 4. CreateDefaultConfig - 71.4% coverage

Missing Test Cases:

Load() Function



Test with valid config file (DONE)



Test with config file not found (uses defaults)



Test with malformed config file (YAML parsing error)



Test with environment variable overrides



Test all environment variable bindings (database, redis, auth)



Test with partial config file (some values from file, some from defaults)

findConfigFile() Function



Test with HELIX_CONFIG env var set (DONE)



Test with HELIX_CONFIG env var pointing to non-existent file



Test with empty HELIX_CONFIG env var



Test searching common locations when env var not set



Test with \$HOME expansion in path

validateConfig() Function



Test valid configuration (DONE)



Test invalid server port (DONE)



Test missing database host (DONE)



Test default JWT secret (DONE)



Test missing database name



Test invalid server port (0 and negative)



Test Redis disabled (should not validate redis fields)



Test Redis enabled with missing host



Test Redis invalid port (0, negative, > 65535)



Test empty JWT secret



Test invalid health check interval (0, negative)



Test invalid max concurrent tasks (0, negative)



Test invalid max retries (negative)



Test invalid max tokens (0, negative)



Test invalid temperature (<0, >2)

CreateDefaultConfig() Function



Test successful creation (DONE)



Test file content validation (DONE)



Test directory creation when parent doesn't exist



Test error when directory creation fails (permission denied)



Test error when file write fails

Estimated Coverage Gain: +10-15%

Implementation Effort: 2-3 hours

Priority 2: Database Package (42.9% coverage)

File: internal/database/database.go

Low-Coverage Functions: 1. InitializeSchema - 0.0% coverage 2. Close - 33.3% coverage 3. HealthCheck - 40.0% coverage 4. GetDB - 66.7% coverage 5. New - 76.5% coverage

Challenge: Database tests require a real PostgreSQL instance.

Recommended Approach: - Use integration tests with Docker Compose - Create test database container in Docker - Run schema initialization tests - Test connection pooling and health checks - Test error scenarios (connection failures, timeout)

Test Setup Required:

```
# docker-compose.test.yml
version: '3.8'
services:
  postgres-test:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: helix_test
      POSTGRES_USER: helix_test
```

```
POSTGRES_PASSWORD: test_password
ports:
  - "5433:5432"
healthcheck:
  test: ["CMD", "pg_isready", "-U", "helix_test"]
  interval: 5s
  timeout: 3s
  retries: 5
```

Missing Test Cases: - [] Test InitializeSchema with fresh database - [] Test InitializeSchema when schema already exists - [] Test InitializeSchema with database connection error - [] Test Close with active connection pool - [] Test HealthCheck with healthy database - [] Test HealthCheck with database down - [] Test HealthCheck with slow database (timeout) - [] Test GetDB with active pool - [] Test New with various connection configurations - [] Test connection pool settings (MaxConns, MinConns, etc.)

Estimated Coverage Gain: +20-25%

Implementation Effort: 4-6 hours (includes Docker setup)

Priority 3: LLM Compression Package (Low 60-70% in some files)

Files: internal/llm/compression/*.go

Low-Coverage Areas: - strategies.go:539: Estimate - 71.4% -
strategies.go:482: Execute - 87.5%

Missing Test Cases: - [] Test compression estimation edge cases - [] Test compression execution with empty messages - [] Test compression with very large message lists - [] Test compression ratio calculation - [] Test token counting with various message types

Estimated Coverage Gain: +3-5%

Implementation Effort: 1-2 hours

Priority 4: Editor Package Edge Cases

Files: internal/tools/editor/*.go

Some edge cases could be added for: - [] Very large file editing (>1MB) - [] Binary file handling - [] Concurrent edit scenarios - [] Rollback on partial failure - [] Unicode and special character handling

Estimated Coverage Gain: +2-3%

Implementation Effort: 2-3 hours

Implementation Timeline

Phase 1 (Week 1)

Goal: Reach 80%+ coverage

1. **Day 1-2:** Config Package Tests
 - Add missing Load() test cases
 - Add findConfigFile() test cases
 - Add validateConfig() edge cases
 - Add CreateDefaultConfig() error cases
2. **Day 3:** LLM Compression Tests
 - Add compression estimation tests
 - Add execution edge cases
3. **Day 4-5:** Database Integration Tests
 - Set up Docker Compose for test database
 - Write InitializeSchema tests
 - Write connection pool tests
 - Write health check tests

Phase 2 (Week 2)

Goal: Reach 85%+ coverage

1. **Day 1-2:** Editor Package Edge Cases
 - Large file tests
 - Concurrent edit tests
 - Error handling tests
 2. **Day 3-4:** Worker Package Tests
 - SSH connection tests (with mock SSH server)
 - Health monitoring tests
 - Task assignment tests
 3. **Day 5:** Review and Fill Gaps
 - Run coverage analysis
 - Identify remaining low-coverage areas
 - Add tests for critical uncovered paths
-

Testing Best Practices

1. Unit Tests (No External Dependencies)

- Use table-driven tests for multiple scenarios
- Mock external dependencies
- Test both success and error paths
- Test edge cases and boundary conditions

2. Integration Tests (With Docker)

- Use Docker Compose for external services
- Clean up test data after each test
- Use random ports to avoid conflicts
- Include health checks for dependencies

3. Test Organization

```
internal/  
  package/  
    package.go  
    package_test.go           # Unit tests  
    package_integration_test.go # Integration tests (build tag)
```

4. Build Tags for Integration Tests

```
//go:build integration  
// +build integration
```

```
package package_test
```

Run integration tests separately:

```
go test -tags=integration ./...
```

Coverage Targets by Package

Package	Current	Target	Priority
internal/config	75.7%	90%+	High
internal/database	42.9%	80%+	High
internal/llm	77.0%	85%+	Medium

Package	Current	Target	Priority
internal/agent	95.0%	95%+	Low (already good)
internal/editor	88.0%	92%+	Medium
internal/worker	72.0%	85%+	High
internal/tools	82.0%	90%+	Medium
Overall	77.0%	80%+	High

Automated Coverage Reporting

CI/CD Integration

Add to `.github/workflows/tests.yml`:

```
name: Tests
```

```
on:
```

```
  push:
```

```
    branches: [ main, develop ]
```

```
  pull_request:
```

```
    branches: [ main, develop ]
```

```
jobs:
```

```
  unit-tests:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v3
```

```
      - name: Set up Go
```

```
        uses: actions/setup-go@v4
```

```
        with:
```

```
          go-version: '1.24'
```

```
      - name: Run unit tests with coverage
```

```
        run: |
```

```
          go test -v -race -coverprofile=coverage.out
```

```
          -covermode=atomic ./...
```

```
          go tool cover -func=coverage.out
```

```
      - name: Check coverage threshold
```

```
        run: |
```

```
COVERAGE=$(go tool cover -func=coverage.out | grep
total: | awk '{print $3}' | sed 's/%//')
echo "Coverage: $COVERAGE%"
if (( $(echo "$COVERAGE < 80" | bc -l) )); then
    echo "Coverage is below 80%"
    exit 1
fi
```

- name: Upload coverage to Codecov
uses: codecov/codecov-action@v3
with:
files: ./coverage.out
flags: unittests
name: codecov-umbrella

integration-tests:

runs-on: ubuntu-latest

services:

postgres:

image: postgres:16-alpine

env:

POSTGRES_DB: helix_test

POSTGRES_USER: helix_test

POSTGRES_PASSWORD: test_password

options: >-

--health-cmd pg_isready

--health-interval 10s

--health-timeout 5s

--health-retries 5

ports:

- 5433:5432

steps:

- uses: actions/checkout@v3

- name: Set up Go

uses: actions/setup-go@v4

with:

go-version: '1.24'

- name: Run integration tests

env:

HELIX_DATABASE_HOST: localhost

HELIX_DATABASE_PORT: 5433


```
HELIX_DATABASE_USER: helix_test
HELIX_DATABASE_PASSWORD: test_password
HELIX_DATABASE_NAME: helix_test
run: |
    go test -v -tags=integration ./...
```

Coverage Badge

Add to README.md:

```
[![codecov](https://codecov.io/gh/helixcode/helixcode/branch/main/graph/badge.svg)](https://codecov.io/gh/helixcode/helixcode)
```

Success Metrics

Coverage Goals

- **Minimum:** 80% overall coverage
- **Target:** 85% overall coverage
- **Stretch:** 90% overall coverage

Quality Metrics

- All critical paths covered (auth, database, task management)
- All error handling paths tested
- All edge cases documented and tested
- Integration tests for external dependencies
- Benchmark tests for performance-critical code

CI/CD Requirements

- All tests pass before merge
 - Coverage doesn't decrease with new code
 - Integration tests run on PR
 - Coverage report generated and uploaded
-

Resources

Tools

- `go test -cover` - Built-in coverage tool
- `go tool cover -html` - Visual coverage report

- `codecov.io` - Coverage tracking and reporting
- `golangci-lint` - Static analysis
- `testify` - Testing assertions and mocks

Documentation

- [Go Testing Documentation](#)
 - [Table-Driven Tests](#)
 - [Testify Documentation](#)
 - [Docker Compose for Testing](#)
-

Next Steps

1. **Complete** - Create coverage improvement plan (this document)
 2. **TODO** - Add missing config package tests (Priority 1)
 3. **TODO** - Set up Docker Compose for database integration tests
 4. **TODO** - Add LLM compression edge case tests
 5. **TODO** - Add database integration tests
 6. **TODO** - Set up CI/CD with coverage checks
 7. **TODO** - Add coverage badges to README
-

Plan Created: 2025-11-07 **Last Updated:** 2025-11-07 **Status:** Active **Owner:**
Development Team